

液晶数值模拟读书会

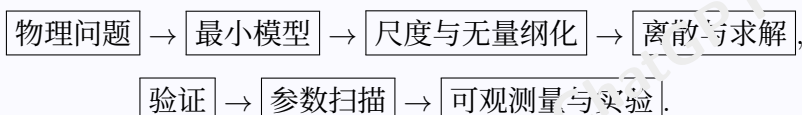
第十一章：From Model to Simulation——怎样独立完成一个液晶数值课题？

Rongxing Xu & ChatGPT

本章路线

前面的章节像是在学习一盒工具：director、Frank elasticity、anchoring、 Q -tensor、外场、chirality、defect、skyrmion、toron、显微镜 forward model……最后一章不再增加新的“工具”，而是学习怎样把它们装进同一个研究问题里。

一个完整的数值课题，最好沿着下面这条路走：



本章的目标不是教会某一个特定程序，而是让读者在遇到一个新的液晶问题时，知道从哪里开始、怎样判断程序是否可信，以及怎样把一张“好看的 simulation 图”变成一个可复现的物理结论。

1 一个数值课题，先从一句物理问题开始

刚开始做数值时，很容易从“我想模拟一个 toron”“我想看看加电以后结构会怎样”开始。这样的句子还不是研究问题，因为程序不知道什么叫“模拟出来了”。真正能交给计算机的问题，必须再向前走一步：**改变什么？测量什么？最后用什么量判断？**

例如，“研究电场对 toron 的影响”可以改写为：在固定 cell thickness d 、pitch p 、anchoring 和材料参数下，逐步提高电压 V ；每个 V 都让体系充分弛豫；记录自由能、toron radius、两个 defect 的间距，以及最后是否回到 uniform state。这样，控制参数、状态变量和输出量就都明确了。

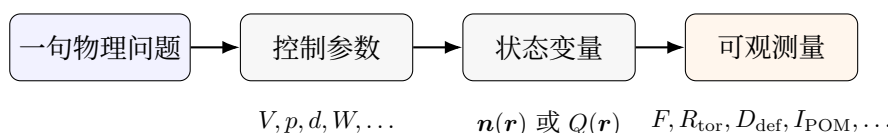


图 1: 把“我想模拟什么”翻译成“我要改变什么、计算什么、测量什么”。这一步做清楚以后，后面的模型选择会轻松很多。

一个很实用的习惯，是在写代码之前先写一页“研究问题卡片”：一句话问题、控制参数、固定参数、边界条件、候选初态、最终输出。它像实验前的实验记录本，可以防止程序越写越复杂，最后却忘了真正想回答什么。

2 第一道选择: director 还是 Q ?

模型不是越复杂越好。更像地图: 去隔壁街区不需要卫星地图, 但如果要穿过山谷, 也不能只带一张地铁图。

如果研究的是平滑 texture、pitch、anchoring、Frederiks transition 或大尺度 toron 形态, 且 defect core 不是问题核心, director theory 往往已经足够。若问题涉及 defect nucleation、annihilation、core size、biaxiality, 或者 order parameter 会显著降低, 则应使用 Q -tensor。只有当流场和 backflow 本身成为问题的一部分时, 才需要继续引入 Beris-Edwards 与 hydrodynamics。

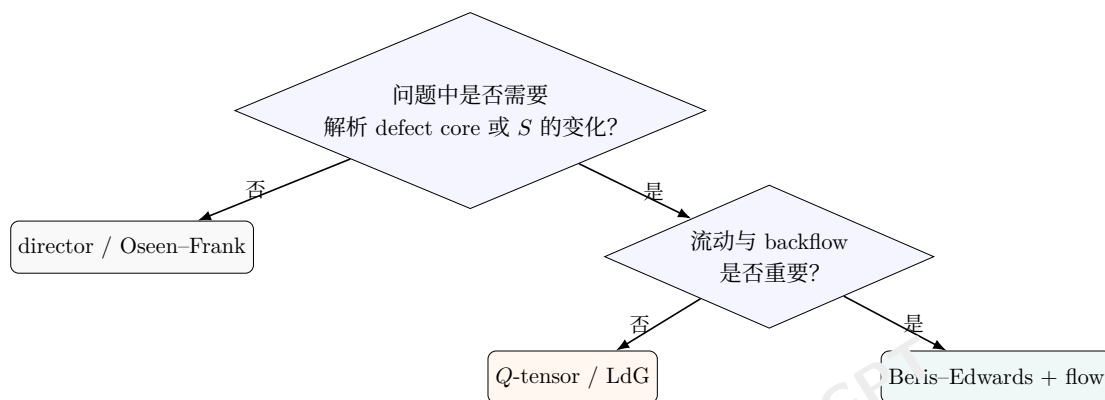


图 2: 一个足够实用的模型选择树。真正的原则是: 用能够回答问题的最简单模型。

同一个课题可以分两层做

很多复杂问题都适合先用便宜模型找到大尺度结构, 再用高分辨率模型局部升级。例如 Chapter 9 的 toron 可以先用 Oseen-Frank 找到 macrotexture, 再把收敛的 director lift 到 Q -tensor 去解析 point-defect core, 这样计算资源花在真正需要的地方。

3 敲代码之前, 先画一张“尺度地图”

液晶问题往往同时有几个长度尺度。比如 chiral cell 里有 pitch p 、cell thickness d ; 有表面 anchoring 时会出现 extrapolation length $\ell_W \sim K/W$; Q -tensor defect core 由 correlation length ξ_Q 控制; 电场还会带来一个 field-induced length。数值网格必须看见问题里最小的相关尺度, 而计算盒必须装得下最大的结构。

可以把这件事记成一句话:

网格由最小尺度决定, 盒子由最大尺度决定。

如果你只关心 smooth director texture, 最小尺度可能是 pitch 的一小部分; 如果必须解析 defect core, 最小尺度就会降到 ξ_Q 。这也是为什么同一个 $10 \mu\text{m}$ 的 cell, director simulation 可以相对轻松, 而 full LdG 可能突然变成一个很重的三维计算。

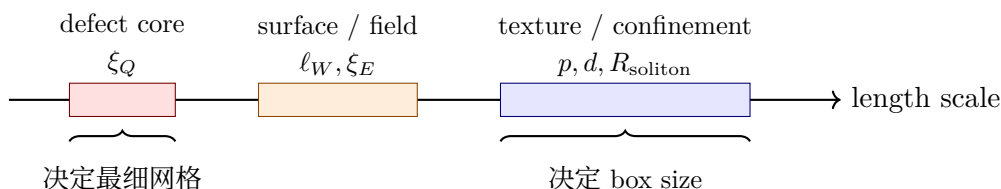


图 3: 数值模拟前先把重要长度尺度排成一条线。不要先定 N_x, N_y, N_z , 再问它们够不够; 顺序应该反过来。

3.1 无量纲化不是为了“看起来简洁”

无量纲化最实际的价值, 是把数值量级拉回到 $O(1)$, 并让真正控制问题的参数组合浮出来。比如用 p 或 d 作为长度单位以后, 系统自然由 $d/p, Wp/K$ 等无量纲组合控制。对 LdG, 选定能量和长度单位后, 也应把代码内部所有量统一成一套 dimensionless convention, 只在输入输出处做 SI 转换。

每个项目最好单独保存一张 conversion table: 代码里的 $x = 1$ 对应多少米, 代码里的 $F = 1$ 对应多少焦耳, 代码里的电场、anchoring、Frank constant 分别怎样换回实验单位。这个小表在几个月以后会非常救命。

4 让自由能、演化方程和边界条件来自同一个故事

一个常见的数值错误, 不是差分写错, 而是“能量用一套模型, 更新公式又用了另一套近似”。最稳妥的做法是先把总自由能写完整: bulk、elastic、surface、field、chirality 各自是否存在; 然后从同一个 F 推出 molecular field, 再离散。

对于 director, 更新需要尊重 $|\mathbf{n}| = 1$; 对于 Q -tensor, 右端必须保持 symmetric traceless。边界也要和能量一致: strong anchoring 是固定边界值, finite anchoring 则应来自 surface energy, 而不是一边写 Rapini-Papular, 一边又把表面节点完全锁死。

一条非常值得坚持的代码规则

能量函数、molecular field 和边界条件最好各自只有一个“source of truth”。如果同一个公式分别散落在三四个脚本里, 半年后几乎一定会出现参数不一致。生产代码可以复杂, 但物理定义应该集中。

4.1 离散也有“物理一致性”

若你用中心差分构造 elastic molecular field, 那么自由能诊断最好使用与之相容的离散形式; 周期边界、Neumann 边界和固定边界也要在差分算子里统一处理。特别是在 defect 或强边界约束问题中, 差分模板若越过了不该跨越的边界, 就会制造并不存在的液晶能量。

数值实现完成后, 先不要急着跑 256^3 。先在很小的 grid 上打印每一项能量、最大 molecular field、边界值、 $|\mathbf{n}| - 1$ 或 $\text{Tr } Q$ 的误差。小系统像显微镜, 更适合抓 bug。

5 初态不是答案，而是向自由能提出问题

液晶自由能往往有很多 local minima。uniform、helix、skyrmion、toron 等结构都可能在相近的控制参数下成为不同的局域极小值。于是“我从 toron seed 出发，最后得到 toron”只说明这个 basin 存在，并不能自动说明它是最低自由能态。

因此，一个独立课题通常需要至少两类初态：一个简单 reference state，再加一个或多个目标结构 seed。若关心相变或稳定边界，可以从参数的一端开始逐步改变控制量，并把上一步的收敛态作为下一步初态，这就是 continuation。正向和反向 continuation 若给出不同分支，往往正是在告诉你体系存在 metastability 或 hysteresis。

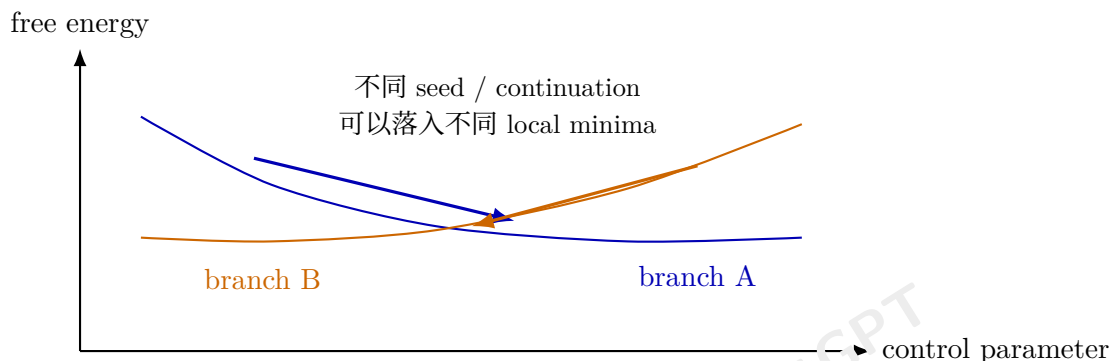


图 4: metastability 的最简单图像。只跑一个 seed，相当于只走一条支路；比较稳定性时需要把竞争分支放在一起看。图为 schematic。

6 先证明程序会做简单题，再让它做难题

数值程序最危险的状态不是“报错”，而是安静地给出一张看起来很合理的错图。最有效的防线，是让程序先完成一个有解析答案的 benchmark。

Chapter 1 已经有一个非常合适的例子。对一维 one-constant director angle，线性 relaxation 满足 diffusion equation；若初态是单一 Fourier mode，振幅应按 $\exp(-Dk^2t)$ 衰减。下面的图不是液晶复杂结构，而是专门用来检查差分和时间推进的最小测试。

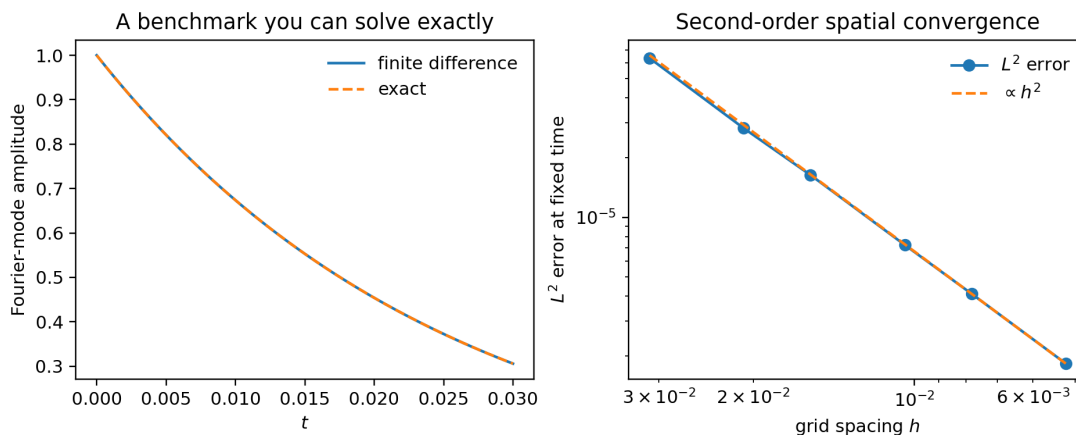


图 5: 由本章附带的小程序生成的 benchmark。左: finite-difference Fourier-mode decay 与解析解重合；右: 固定时间的 L^2 error 随 grid spacing 近似按 h^2 下降，符合二阶中心差分预期。只有通过这类简单测试，才值得把同一套算子交给复杂 texture。

一个新的 solver, 建议按下面的顺序长大: 先验证均匀态的能量; 再验证一个一维解析 profile; 再做二维; 最后才进三维。每增加一层复杂度, 都保留前一层测试。真正成熟的研究代码, 应该能够随时重新运行这些 benchmark, 而不是只在开发第一周测试一次。

7 “收敛”不是跑了很多步

程序跑到 20000 步并不等于收敛。一个静态 free-energy minimization 至少要同时看三件事: 自由能是否进入平台、最大 molecular field 是否足够小、你真正关心的结构量是否不再漂移。例如 toron 可以看半径与 defect separation, cholesteric 可以看 pitch, Frederiks transition 可以看最大 tilt angle。

常用的停止条件可以写成相对能量变化很小, 同时 residual 很小; 具体阈值依模型和单位而定, 关键是把阈值写进程序并输出日志, 而不是凭肉眼判断“差不多了”。

四种不同的“收敛”

iteration convergence: 同一网格上不再随迭代变化; **time-step convergence:** 减小数值步长后结果不变; **grid convergence:** 减小空间网格后宏观结果稳定; **box-size convergence:** 扩大计算区域后结果不再受边界挤压。
真正可信的结果至少要知道自己通过了哪几种, 而不是把它们统称为“收敛”。

对 defect 问题, grid convergence 尤其重要。若一个 defect core 只有一两个格点宽, 位置、能量甚至是否 annihilate 都可能被 lattice artifact 决定。此时更细的网格不是“让图更漂亮”, 而是在改变模型是否真正解析了那段物理。

8 从一张漂亮图, 走到一个物理结果

完成 relaxation 后, 第一张图通常是 director rods、 S colormap 或等值面。它们适合看结构, 但论文结论通常需要更可比较的量。一个好的 observable 应该随着参数变化给出数字, 而不仅仅是“看起来变大了”。

例如: Frederiks transition 可以测中心 tilt; cholesteric 可以测 pitch; skyrmion/toron 可以测半径、 N_{sk} 与 defect separation; Chapter 10 还教会我们把 director/ Q 继续送进 POM/FCPM forward model, 与实验图像直接比较。

参数扫描时, 建议让程序自动输出一个整洁的数据表: 每一行是一组控制参数, 每一列是一个 observable, 再额外记录初态、最终状态标签、总自由能、迭代步数与 convergence flag。这样后面的 phase diagram、error bar 和统计分析会变得非常自然。

8.1 比较状态时, 要比较同一个物理问题

比较两种结构的自由能时, box、网格、材料参数、边界条件和外场必须完全一致。若一个状态用 128^3 、另一个用 96^3 , 或者一个把 surface energy 算进总能、另一个没有, 得到的差值没有意义。自由能差往往比各自总能小很多, 所以“公共项是否一致”尤其重要。

9 把代码组织成一个以后还能读懂的项目

当脚本超过几百行，最值得做的不是继续加注释，而是让不同职责分开。一个够用的项目结构可以是：

```
project/
  config.py          # 参数与单位
  physics.py         # free energy / molecular field
  boundaries.py      # anchoring, fixed/periodic boundaries
  initial_states.py  # uniform, helix, skyrmion, toron...
  solver.py          # FIRE / gradient flow / time integrator
  diagnostics.py     # F, residual, radius, defects...
  imaging.py         # POM / FCPM forward model
  plot.py            # 只负责画图
  run.py             # 一个清楚的入口
  tests/             # analytic benchmarks
  results/           # 每次 run 的参数 + 数据 + 日志
```

这种分法的好处是，改变一个画图习惯不会碰到 solver，换一个 initial state 不需要复制整套 free-energy code。更重要的是，每次 simulation 应把完整参数一起保存。半年以后只剩一个 state_final.npy 而找不到它对应的 K_{ii} 、 p 、 W 和网格，几乎等于没有保存结果。

建议每次 run 自动保存的 metadata

至少包括：代码版本或日期、grid size、box size、dimensionless/SI conversion、所有材料参数、边界条件、初态名称、solver 参数、停止条件、随机种子（若有）、最终能量、residual 和主要 observable。最好让程序自动写成 JSON/YAML，而不是依赖手工记忆。

10 程序不对时，先看哪里？

复杂三维 simulation 出问题，最有效的方法不是同时改十个参数，而是先判断错误属于哪一层。

症状	常见来源	第一步检查
自由能持续上升或 NaN	步长过大、符号错误、mask/边界更新错误	缩小系统与步长；单独检查每项能量和 $\max H $
结构总是贴到边界	box 太小、周期镜像相互作用	扩大 box，比较结构尺寸是否变化
defect 一换网格就移动/消失	core 未解析、lattice pinning	画 S core；做至少两到三档 grid refinement
目标结构总是消失	seed 不合适，或该参数下确实没有 local minimum	先验证 seed；再做 continuation 与竞争态比较
图像和实验不像	光学参数、orientation、pixel/PSF 未对齐	先比较结构 observable，再校准 Chapter 10 的 forward model

调试时尽量一次只动一个东西。若同时换网格、时间步、材料参数和初态，即使结果“好了”，也不知道究竟是哪一步起作用。

11 最后一个训练：把一个问题独立做成小课题

这套读书会最后不再给一个固定答案。每位读者选择一个自己感兴趣的问题，先写一页 proposal，再做一个最小但完整的 numerical project。题目可以很小，例如：anchoring strength 怎样移动 Frederiks threshold； d/p 怎样改变 chiral texture；外场怎样改变一个已有 toron；不同初态是否会落入不同的 metastable branches；或者把某个已知三维结构转成 POM/FCPM 图像。

项目不以“模型越复杂”评分，而看下面四件事是否完整：问题是否明确；程序是否通过 benchmark；关键结果是否经过 convergence check；结论是否由 observable 和对照支持。

一个合格的小课题最终应该交什么？

一页物理问题与模型说明；一份可以从头运行的代码；至少一个解析或已知 benchmark；一组 convergence test；一张主结果图加一张定量曲线；完整参数表；以及一段能够回答“这个结果为什么可信”的讨论。

如果还可以把模拟结果通过 POM/FCPM forward model 与实验图像比较，或者用两种不同初态排除 metastability 误判，那么这个小课题已经非常接近真正的研究工作流。

12 本章总结：数值模拟是一条证据链

一个好的液晶 numerical project，不是“把方程写进 GPU 然后得到一张彩色图”。它更像一条从物理问题出发、逐层建立证据的链条：模型为什么足够；尺度为什么分辨得了；边界和初态为什么合理；离散程序为什么正确；结果为什么收敛；竞争状态有没有比较；最终 observable 怎样连接到实验。

当这条链条完整时，即使最后得到的是“目标结构并不稳定”，它仍然是一个有价值的数值结论。反过来，如果链条缺了几环，再漂亮的 3D rendering 也只能算一次计算演示。

到这里，这套读书会从最开始的 nematic order，走到了能够独立设计一个数值课题。真正下一步要学的，不再是更多公式，而是在一个具体研究问题里反复练习这套工作流。

参考资料

参考文献

- [1] P. G. de Gennes and J. Prost, *The Physics of Liquid Crystals*, 2nd ed., Oxford University Press (1993).
- [2] D. M. Sussman and D. A. Beller, “Fast, scalable, and interactive software for Landau-de Gennes numerical modeling of nematic topological defects,” *Frontiers in Physics* **7**, 204 (2019).
- [3] T. A. Davis and E. C. Gartland Jr., “Finite element analysis of the Landau-de Gennes minimization problem for liquid crystals,” *SIAM J. Numer. Anal.* **35**, 336–362 (1998).
- [4] open-Qmin documentation and source code, an open-source lattice-discretized Landau-de Gennes simulation framework.